jStack

TECHNICAL ASSESSMENT · ROUND 1

Flutter Developer Assignment

jWallet Lite — Digital Wallet Mobile Application

Position	Flutter Developer
Experience Required	2–5 years
Job Location	Bhopal, Madhya Pradesh
Assessment Type	Take-home technical assignment
Estimated Effort	5–7 hours
Submission Deadline	Within 48 hours of receiving the assignment
Total Marks	100

01 Assignment Overview

At jStack, we build digital financial infrastructure — payment platforms, digital wallets, fintech applications, and enterprise financial solutions.

As part of our technical evaluation, you will design and develop **jWallet Lite**, a lightweight digital wallet mobile application built with **Flutter and Dart** and integrated with a **Mock REST API backend that you develop yourself**.

The assignment evaluates your practical skills in Flutter development, UI implementation, state management, REST API integration, application architecture, and problem-solving.

Expected Deliverables

- A functional Flutter mobile application
- A lightweight Mock REST API backend
- Integration between the Flutter application and the backend APIs
- Appropriate error handling and validation
- Clean, maintainable, and well-structured source code
- Basic unit and widget tests
- Technical documentation and a demonstration video

Important Using static JSON files inside the Flutter application as a substitute for backend integration is not permitted.

02 Technology Requirements

Mobile Application

Component	Technology
Framework	Flutter
Programming Language	Dart
State Management	Riverpod / Bloc / Provider
Architecture	MVVM / Clean Architecture
API Integration	Dio / HTTP
Navigation	GoRouter / Flutter Navigator
Local Storage	SharedPreferences / Flutter Secure Storage
UI	Material Design 3
Testing	Flutter Test / Mocktail

You may use alternative Flutter packages, provided you can explain your technical decisions.

Mock API Backend

Component	Technology
Backend	Node.js + Express.js (preferred)
API Architecture	REST
Data Format	JSON
Data Storage	In-memory / JSON file / SQLite
Authentication	Mock Bearer token
API Testing	Postman / Bruno / cURL

The backend should be lightweight and run independently. No cloud deployment or production database is required.

03 Functional Requirements

Module 1 — Login & Authentication

Screen 1: Login

Build a login screen that includes:

- jWallet Lite application branding
- Email address input
- Password input with show/hide toggle
- Login button
- Form validation
- Loading indicator
- Error message for invalid credentials

Demo Credentials

Email	`demo@jstack.co`
Password	`Demo@123`

Expected Workflow

1. User opens the application.
2. User enters login credentials.
3. Application calls the Login API.
4. Backend validates the credentials.
5. Backend returns an authentication token.
6. Application stores the token securely.
7. User is redirected to the wallet dashboard.

Additional Requirements

- Implement logout functionality
- Handle invalid credentials
- Persist the login session across application restarts
- Handle API timeouts and connectivity errors

Module 2 — Wallet Dashboard

Screen 2: Home Dashboard

Design a clean, modern digital wallet dashboard with the following components:

Section	Required Elements
User Information	User name · Profile avatar or initials · Welcome message
Wallet Information	Available balance · Currency · Masked wallet account number
Quick Actions	Send Money · Add Money (UI only) · Transaction History
Recent Transactions	Latest five transactions, showing amount, type, date, and status

Sample Wallet Data

Field	Value
User Name	Demo User
Wallet ID	WALLET10001
Available Balance	₹25,450.75
Currency	INR
Account Number	XXXX XXXX 4589

Functional Requirements

- Fetch the wallet balance from the backend API
- Fetch recent transactions from the backend API
- Implement pull-to-refresh
- Display loading and error states
- Update the wallet balance after successful transactions

> **Important** The wallet balance must be fetched from the backend. It must not be hardcoded in the Flutter application.

Module 3 — Send Money

Screen 3: Money Transfer

Create a money transfer screen with the following fields:

Field	Validation
Beneficiary Name	Required
Mobile Number	Valid 10-digit number
Transfer Amount	Greater than zero
Remarks	Optional

Functional Workflow

1. User taps Send Money.
2. Application opens the transfer form.
3. User enters beneficiary details and amount.
4. Application validates the form.
5. User reviews the transfer confirmation.
6. Application calls the Transfer API.
7. Backend validates the transfer.
8. Backend updates the wallet balance and transaction history.
9. Application displays the transaction result.

Success Screen

Display the following:

- Success message
- Transaction reference number
- Amount transferred
- Beneficiary name
- Transaction date and time
- Back to Dashboard button

Mandatory Validation

The application must handle:

- Insufficient wallet balance
- Invalid beneficiary mobile number
- Zero or negative transfer amount
- Empty mandatory fields
- Duplicate transfer submission
- API failure

> **Business Rule** The backend is solely responsible for balance validation and deduction. The Flutter application must not independently modify the authoritative wallet balance.

Module 4 — Transaction History

Screen 4: Transactions

Build a transaction history screen. Each transaction should display:

- Transaction reference number
- Beneficiary/sender name
- Transaction amount
- Transaction type (Credit/Debit)
- Transaction status

- Transaction date and time

Functional Requirements

- Fetch transaction history through the API
- Display transactions in descending chronological order
- Search by transaction reference or beneficiary name
- Filter transactions by Credit/Debit
- Display an empty state when no transactions exist
- Display loading and error states

Screen 5: Transaction Details

When a user selects a transaction, display:

- Transaction ID
- Transaction type
- Amount
- Beneficiary/sender
- Date and time
- Transaction status
- Remarks (if available)

04 Mock REST API Development

Mandatory You must build your own Mock REST API backend and integrate it with the Flutter application. The backend must run independently of the mobile app.

Required API Endpoints

Method	Endpoint	Description
POST	/api/v1/auth/login	Authenticate user
GET	/api/v1/users/profile	Fetch user information
GET	/api/v1/wallet/balance	Fetch wallet balance
GET	/api/v1/transactions	Fetch transaction history
POST	/api/v1/transactions/transfer	Simulate money transfer
GET	/api/v1/transactions/:id	Fetch transaction details

API 1 — Login POST /api/v1/auth/login

Request

```
{
```

```
  "email": "demo@jstack.co",
  "password": "Demo@123"
}
```

Successful Response

```
{
  "success": true,
  "data": {
    "accessToken": "mock_access_token_123",
    "user": {
      "id": "USR10001",
      "name": "Demo User",
      "email": "demo@jstack.co"
    }
  }
}
```

Invalid credentials must return **HTTP 401**.

API 2 — Wallet Balance GET /api/v1/wallet/balance

Response

```
{
  "success": true,
  "data": {
    "walletId": "WALLET10001",
    "currency": "INR",
    "availableBalance": 25450.75,
    "maskedAccountNumber": "XXXX XXXX 4589"
  }
}
```

This endpoint must require a valid mock authentication token.

API 3 — Transaction History GET /api/v1/transactions

Response

```
{
  "success": true,
  "data": [
    {
      "id": "TXN10001",
      "type": "DEBIT",
      "amount": 1500.00,
      "currency": "INR",
      "beneficiaryName": "Rahul Sharma",
      "status": "SUCCESS",
      "createdAt": "2026-10-20T10:30:00Z"
    },
    {
      "id": "TXN10002",
      "type": "CREDIT",
      "amount": 5000.00,
      "currency": "INR",
      "beneficiaryName": "Amit Kumar",
```

```
      "status": "SUCCESS",
      "createdAt": "2026-10-19T14:15:00Z"
    }
  ]
}
```

The backend must provide at least **10 sample transactions**.

API 4 — Money Transfer POST /api/v1/transactions/transfer

Request Headers

```
Authorization: Bearer mock_access_token_123
Idempotency-Key: unique-request-id
Content-Type: application/json
```

Request Body

```
{
  "beneficiaryName": "Rahul Sharma",
  "mobileNumber": "9876543210",
  "amount": 1500.00,
  "remarks": "Payment for services"
}
```

Successful Response

```
{
  "success": true,
  "data": {
    "transactionId": "TXN10003",
    "status": "SUCCESS",
    "amount": 1500.00,
    "currency": "INR",
    "availableBalance": 23950.75
  }
}
```

Backend Business Logic

The backend must:

- Validate the authentication token
- Validate beneficiary details and transfer amount
- Check the available wallet balance
- Reject transfers that exceed the available balance
- Deduct the amount after successful validation
- Generate a unique transaction reference
- Add the transaction to the transaction history
- Return the updated wallet balance
- Prevent duplicate deductions for requests that reuse the same idempotency key

> **Idempotency** For repeated requests with the same key and an identical payload, return the original transfer result without deducting the balance again.

Expected Error Handling

Scenario	HTTP Status
Successful request	200
Invalid request data	400
Invalid credentials/token	401
Transaction not found	404
Insufficient balance	409
Unexpected server error	500

Use a consistent JSON error structure across all endpoints.

05 Flutter Application Architecture

Implement a maintainable application structure. The recommended layering is:

Layer	Responsibilities
Presentation	Screens · Widgets · State management · UI validation · Navigation
Domain	Business entities · Use cases · Repository interfaces
Data	API services · Request/response models · Repository implementations · Secure local storage

Architecture Expectations

- Avoid placing API calls directly inside UI widgets
- Separate business logic from presentation logic
- Use reusable UI components
- Handle asynchronous operations correctly
- Maintain consistent application state
- Avoid hardcoded API responses inside Flutter screens
- Never commit credentials or secrets

You are free to choose the exact folder structure.

06 UI/UX Expectations

The application should demonstrate sound mobile UI development practices.

Design Requirements

- Clean, professional fintech interface
- Consistent typography and spacing
- Reusable components
- Appropriate iconography
- Clear navigation
- Responsive layouts across common Android screen sizes
- Loading indicators
- Error and empty states
- User-friendly validation messages

Minimum Required Screens

No.	Screen
1	Login
2	Wallet Dashboard
3	Send Money
4	Transfer Confirmation
5	Transfer Success/Failure
6	Transaction History
7	Transaction Details

You may use your own design approach — a pixel-perfect implementation against a supplied design is not required.

07 Testing Requirements

Implement basic automated tests covering at least:

- Login form validation
- Transfer amount validation
- Insufficient balance handling
- Successful transfer state update

You may use mocked repositories or HTTP clients. Additional widget and integration tests will be viewed favourably.

08 Bonus Features (Optional)

If time permits, you may implement additional functionality, such as:

- Dark mode
- Transaction pagination
- Date-range filtering for transactions
- Biometric authentication
- Improved animations
- Offline transaction caching
- Additional widget tests
- Dependency injection
- Accessibility improvements

Note Bonus features are not required to qualify. Prioritise completing the mandatory functionality correctly.

09 Submission Instructions

Submit the following:

Item	What to Provide
A. Source Code	A GitHub repository, accessible to the jStack evaluation team, containing the Flutter app source, Mock API backend source, README, and sample environment configuration (if applicable)
B. Application Build	A debug or release APK installable on an Android device
C. API Documentation	A Postman collection, Bruno collection, or OpenAPI specification
D. Screenshots	Screenshots of the major application screens
E. Demonstration Video	A 2–3 minute screen recording (see below)
F. README	Setup and architecture documentation (see below)

Demonstration Video Must Show

- Login
- Wallet dashboard
- A successful money transfer
- The updated wallet balance
- Transaction history
- At least one validation or error scenario

README Must Explain

- Prerequisites and Flutter version
- Dependencies
- Application setup
- Mock backend setup
- API base URL configuration
- Android emulator / physical device connectivity
- Steps to run tests
- Architecture decisions
- Known limitations

The evaluation team should be able to run both the application and the backend by following the README alone.

10 Evaluation Criteria

Criteria	Max Marks
UI/UX Implementation	15
Authentication & Session Management	10
Wallet Dashboard	10
Money Transfer Workflow	15
Transaction History	10
Mock API Development	15
API Integration & Error Handling	10
Architecture & Code Quality	10
Testing & Documentation	5
Total	**100**

What the Evaluation Team Will Assess

Dimension	Key Question
Functionality	Does the application work as expected?
Architecture	Is the code modular, structured, and maintainable?
API Integration	Does the app communicate correctly with the candidate-developed backend?
State Management	Are loading, success, and error states handled appropriately?

Dimension	Key Question
Code Quality	Is the implementation readable, reusable, and testable?
Problem-Solving	Are edge cases and validation scenarios handled correctly?
Documentation	Can another developer run and understand the project?

11 Important Guidelines

- The assignment must be developed using Flutter and Dart.
- Developing the Mock REST API backend is mandatory.
- All wallet and transaction data must come from backend APIs.
- No real payment gateway integration is required.
- Do not use real customer data or financial credentials.
- Third-party open-source packages are permitted.
- AI-assisted development tools are permitted, but you must understand and be able to explain all submitted code.
- Disclose any major AI-generated components in your README.
- The assignment must be your own implementation.
- Incomplete submissions may still be evaluated if their limitations are documented.

12 Submission Deadline

Deadline	48 hours from the time the assignment is shared
Expected Effort	Approximately 5–7 hours

You are not expected to spend the full 48-hour window on development.

Please submit your repository, APK, documentation, and demonstration video through the communication channel specified by the recruitment team.

Thank you for your interest in joining jStack. We look forward to reviewing your work.

jStack | Build · Deploy · Scale